

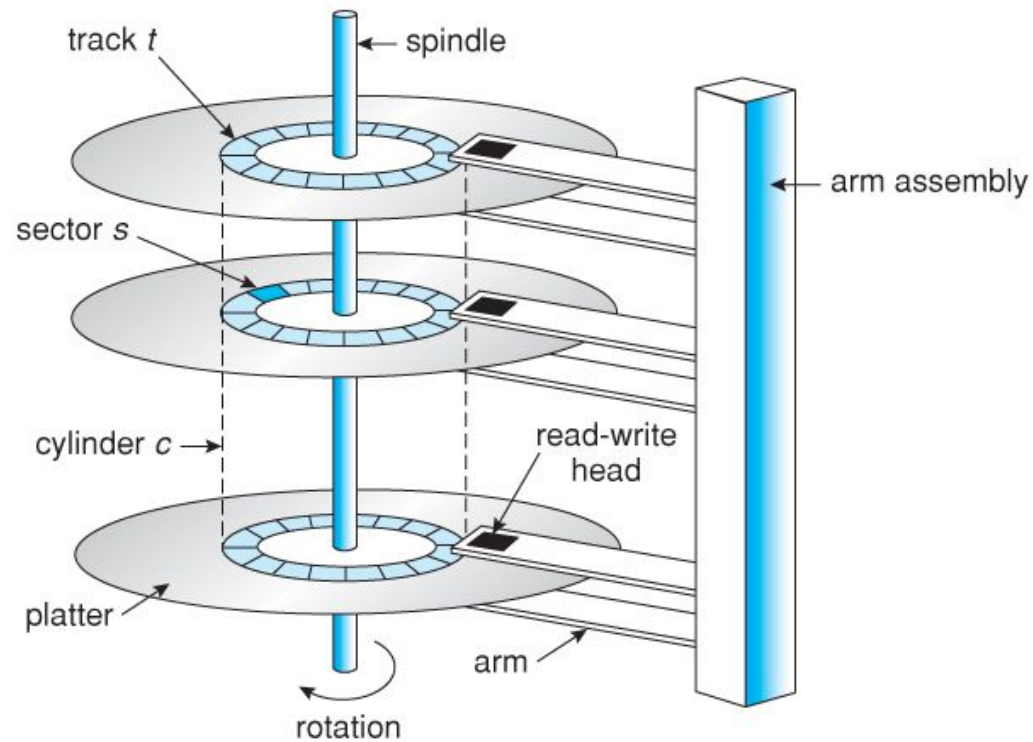
Discos

Algoritmos de planificación y
mecanismos de redundancia



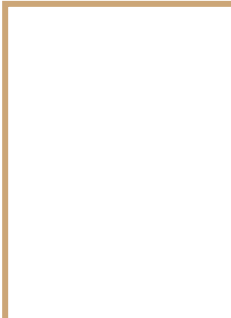
Discos

- Un **sector** es la unidad mínima en un disco duro
- Leo y escribo de a **sectores**
- Los sectores pertenecen a **pistas** (*tracks*)
- Un conjunto de pistas en varios platos (*platters*) es un **cilindro**
- El brazo sube y baja para cambiar entre cilindros
- El disco gira para cambiar entre sectores



Conceptos

- Queremos mejorar:
 - Tiempo de acceso
 - Bandwidth (ancho de banda teórico)
- Tiempo de acceso:
 - **Seek-time** (tiempo de búsqueda): Es el tiempo que tarda el brazo en colocarse en el **cilindro** correspondiente
 - **Latencia de rotación** : Es el tiempo que tarda el disco en rotar hasta el **sector** correspondiente
- Bandwidth:
 - Cantidad de bytes transferidos / (Tiempo en satisfacer todas las requests)
 - ej 2 Megabytes totales en 1 segundo = 1MB/s
- Van a cambiar según el algoritmo que use y las requests que reciba



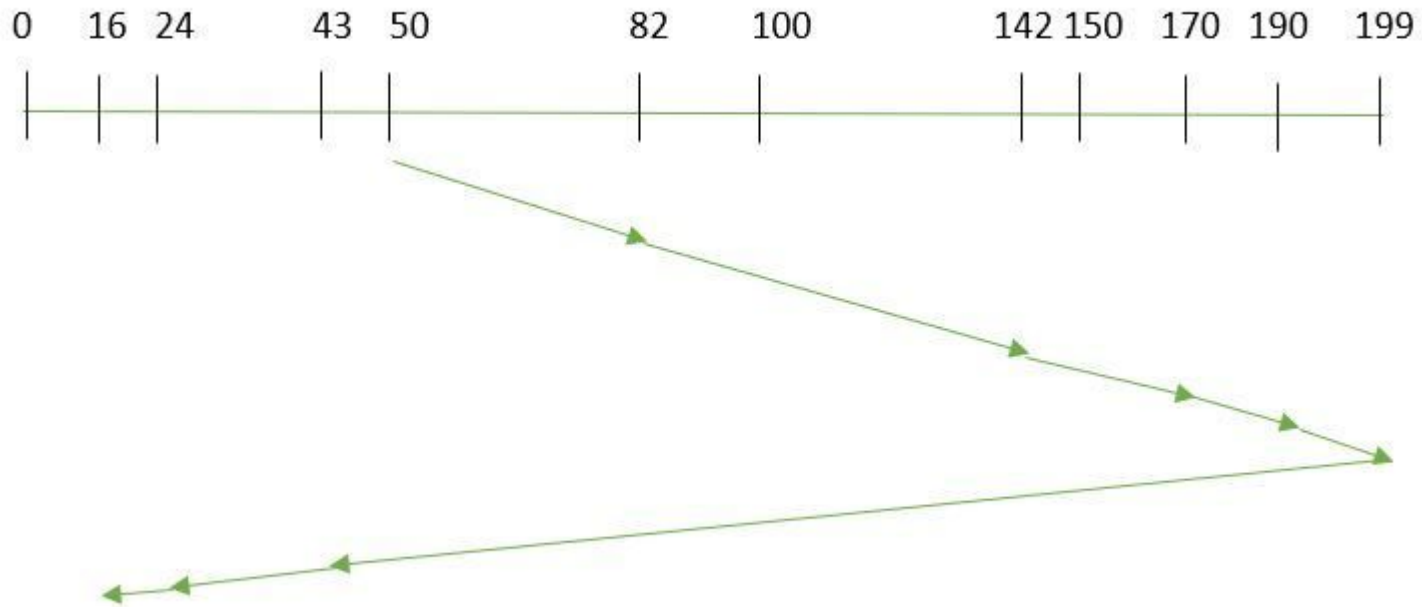
Algoritmos de Planificación



Algoritmo	Descripción	Ventajas	Desventajas
RSS Random Scheduling	Atiende de manera aleatoria los requests, usado para simulaciones		
LIFO Last in First Out	Atiendo siempre el último request	Pocos recursos, maximiza la localidad de requests	Produce <i>starvation</i> de requests viejas si vienen varias nuevas juntas
SSTF Shortest Seek Time First	El que queda más cerca primero	Mejora el tiempo de atención promedio de requests	Overhead de hacer el cálculo de cuál está más cerca. Alta varianza en la atención, fuera de orden.
SCAN Recorro el disco de lado a lado	Voy moviendo el brazo de izquierda a derecha y atendiendo los que encuentro en el camino	Poca varianza entre tiempo de atención de requests.	Si ya atendí un request a un sector y tengo que volver a él, voy a tardar mucho hasta que el brazo vuelva a pasar
C-SCAN Scan circular	Igual que SCAN solo que solo lo hago a la ida y vuelvo nuevamente al comienzo	Mejor tiempo de espera que SCAN al evitar volver pasar por encima de un área recientemente escaneada.	Sigo yendo hasta el final cuando tal vez ni hacía falta

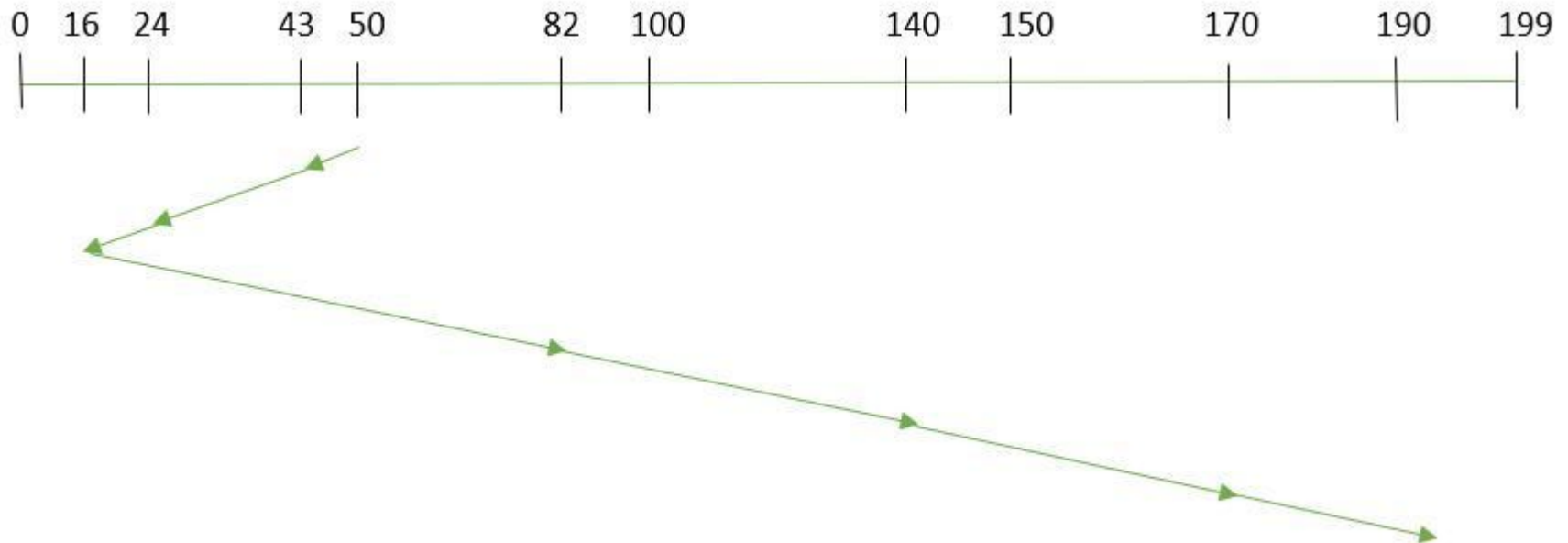
SCAN

- Orden de requests: [82,170,43,140,24,16,190]
- Posición inicial: [50]



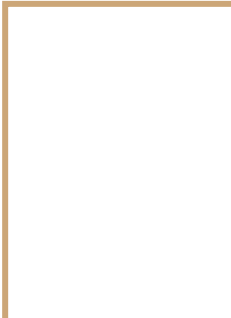
SSTF

- Orden de requests: [82,170,43,140,24,16,190]
- Posición inicial: [50]



- Ejemplo de cálculo de Bandwidth:
 - Sectores = 1024 bytes
 - Tiempo total = 1 segundo
 - Bandwidth = 7.168 bps

Algoritmo	Descripción	Ventajas	Desventajas
LOOK Cómo Scan sin ir hasta el final	Recorro de izquierda a derecha pero en vez de ir al final del disco voy hasta el request mas lejano de cada lado	Mejora performance en todo sentido en comparación con SCAN	Misma que SCAN pero menos grave ya que no voy hasta el final sino hasta el último request
C-LOOK Look circular	Igual que Look solo que en vez de volver atendiendo requests solo lo hago a la ida	Mismas que CSCAN	No dice cómo manejar las colas de requests que van llegando simplemente apila requests y los atiende como puede.



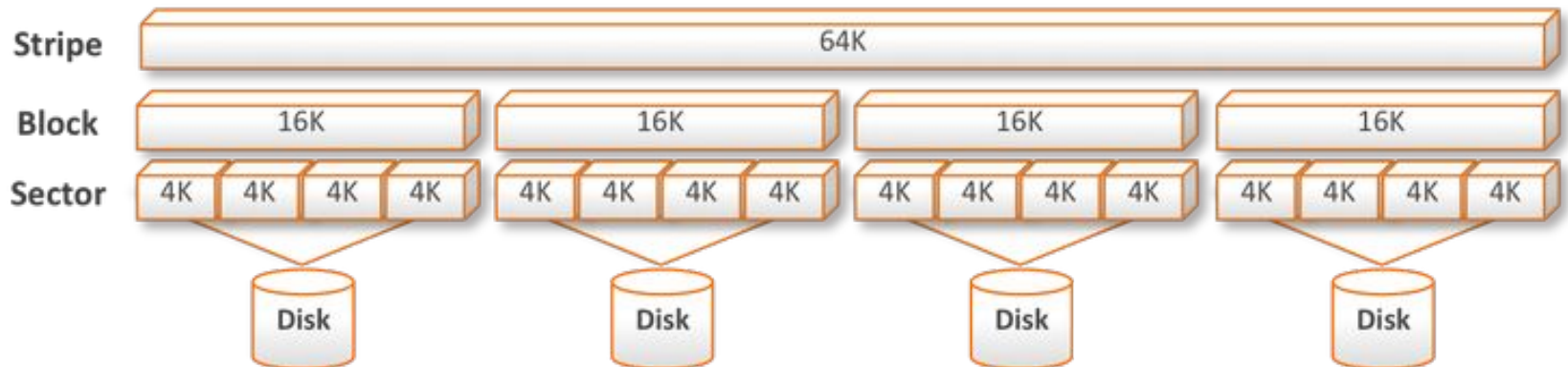
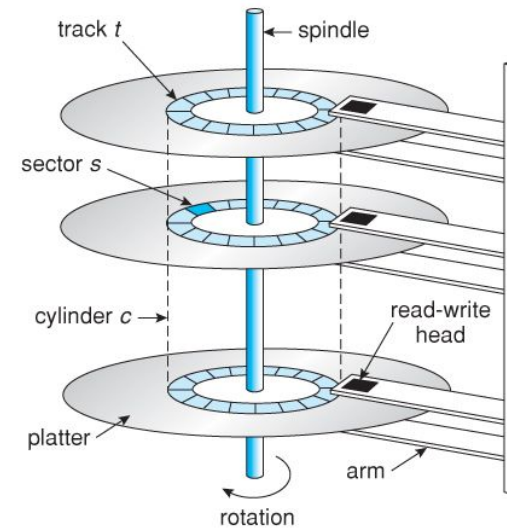
R.A.I.D

Redundant Array of Independent
Discs



Discos

- Un **bloque** es uno o múltiples **sectores** “lógicos”
- El sistema operativo puede decidir qué tamaño van a tener
- Un bloque es la unidad mínima de escritura y lectura “lógica”
- También puedo repartir varios bloques en varios discos en **bandas** (stripes)
- De esta manera gano en performance
- Mayor probabilidad de pérdida de datos => necesito redundancia



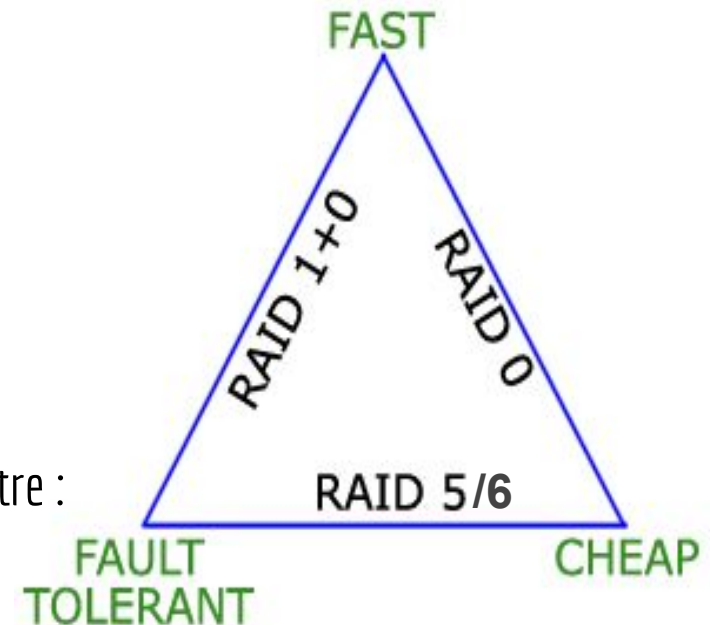
RAID

- Es una manera de virtualización de almacenamiento que permite conectar múltiples discos (físicos) y simular como si fuera un solo disco (lógico)
- La idea es lograr capacidad, performance o disponibilidad mayores a la que permite un solo disco, combinando varios.

- **NO ES UN BACKUP**

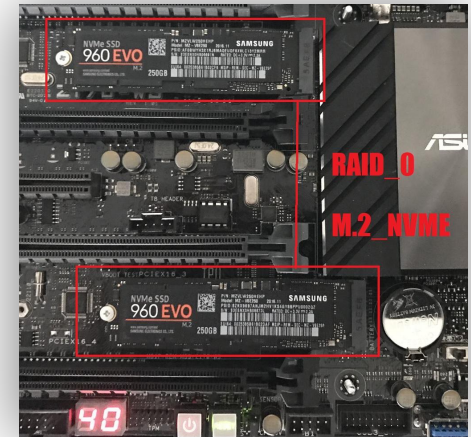
- Cada esquema de **RAID** (nivel) provee un balance diferente entre :

- Capacidad
- Disponibilidad
- Performance
- Costo

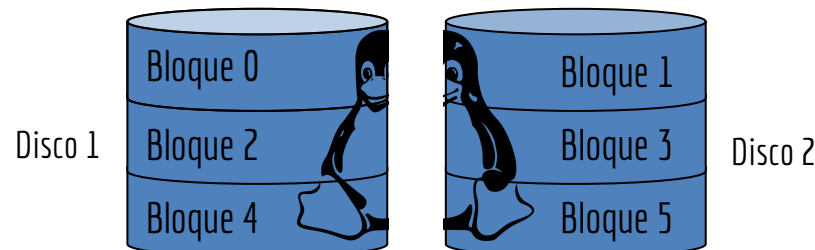


RAID 0 (Bandas de bloques)

- No Redundante
- Alta performance
- Lectura y escritura paralela en bandas (striping) de los bloques en cada disco para mas velocidad
- La **capacidad** y **velocidad** de **n** discos ($1\text{tb} + 1\text{tb} = 2\text{tb}$)
- Si un disco falla, se pierde todo el array
- **Aplicación Normal** : Alto rendimiento para datos no críticos. Por ejemplo: Cache.



Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
N*X	0%	0 Discos



Unidad 5 (Sistema Entrada/Salida)

* N es la cantidad de discos en RAID

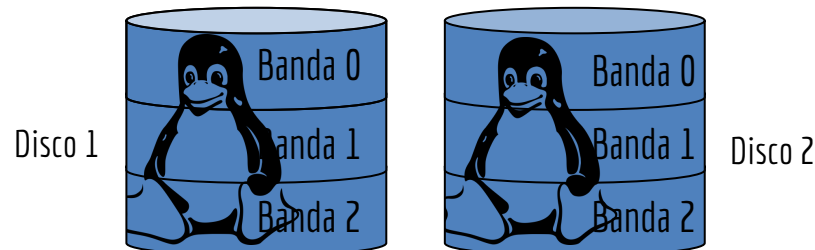
* X es la velocidad de un solo disco

RAID 1

- Redundante
- Lectura paralela en copia espejo + redundancia
- La **capacidad** y **velocidad** de 1 disco ($1\text{tb} + 1\text{tb} = 1\text{tb}$)
- No escala con más discos, solo agrega redundancia y Lectura
- Puede pensarse como el subconjunto de **RAID 10** más pequeño posible
- **Aplicación Normal:** Pocos discos y datos críticos x ej Backup casero



Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
NX X	N-1 Discos	N-1 Discos



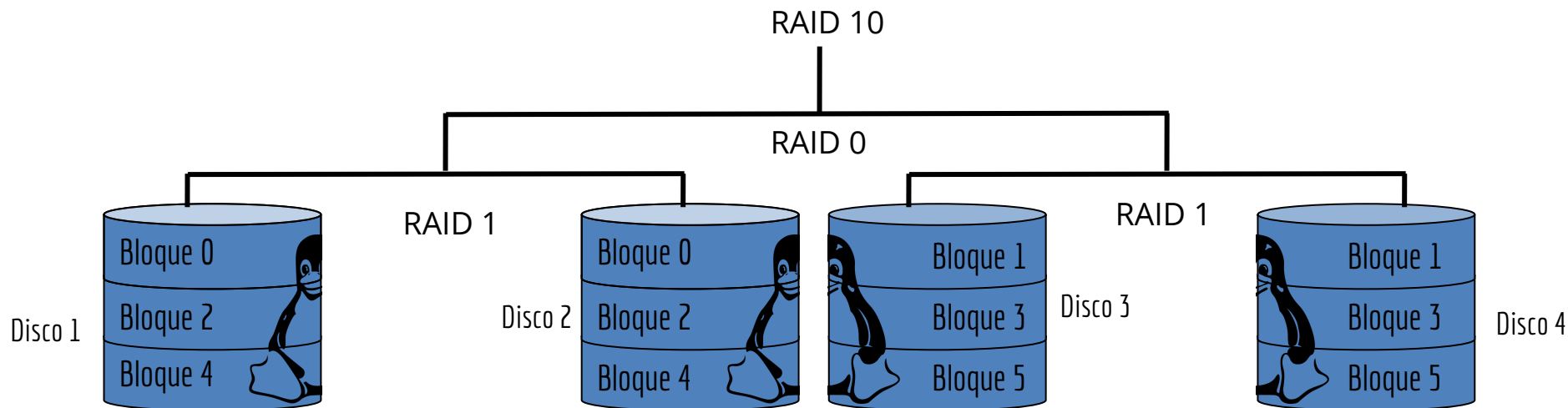
RAID 1+0 (Striping de Espejos)

- Lectura y escritura paralela en bandas (striping) de bloques
- Copia espejo por cada disco
- **Costoso** pero efectivo
- Balance entre buen rendimiento y un mínimo de persistencia, sin recurrir a paridad
- Muchos casos de uso



Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso) *
NX NX/2	-50%	1 Disco

* en 1+0 se puede perder 1 disco por subarray en el mejor de los casos



* N es la cantidad de discos en RAID

* X es la velocidad de un solo disco

Unidad 5 (Sistema Entrada/Salida)

14

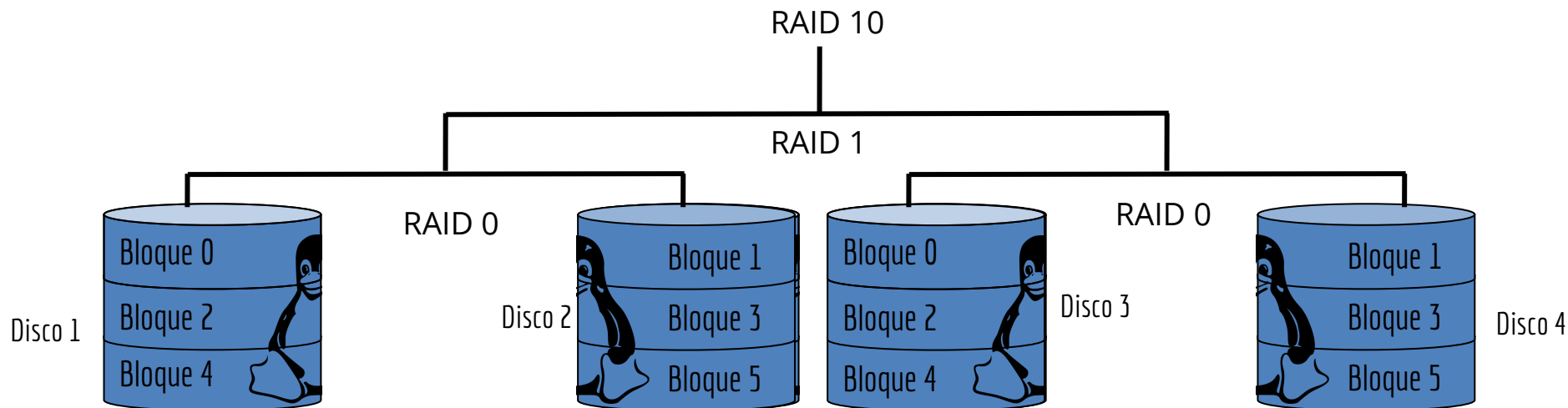
Mínimo 4 discos

RAID 0 +1 (Espejos de stripes)

- Lectura y escritura paralela en bandas (striping) de datos
- Copia espejo por cada disco
- **Costoso** pero efectivo
- Balance entre buen rendimiento y un mínimo de persistencia, sin recurrir a paridad



Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
$NX \mid NX/2$	-50%	1 Disco



* N es la cantidad de discos en RAID

* X es la velocidad de un solo disco

Unidad 5 (Sistema Entrada/Salida)

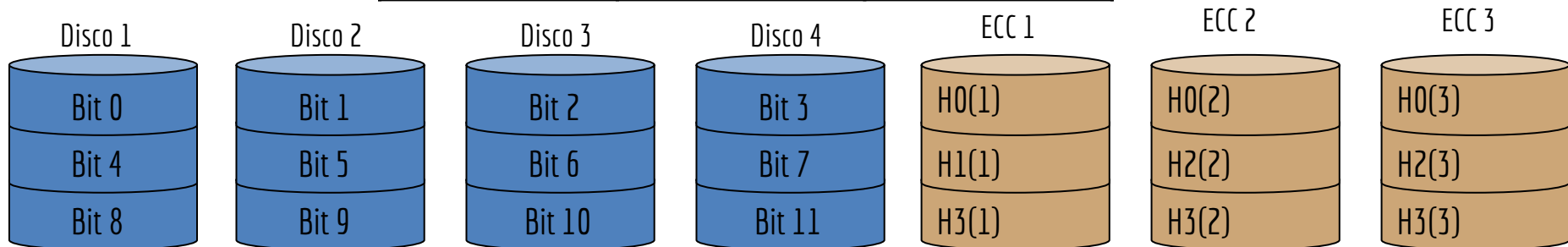
15

Mínimo 4 discos

RAID 2 (Striping de Bits con código Hamming)

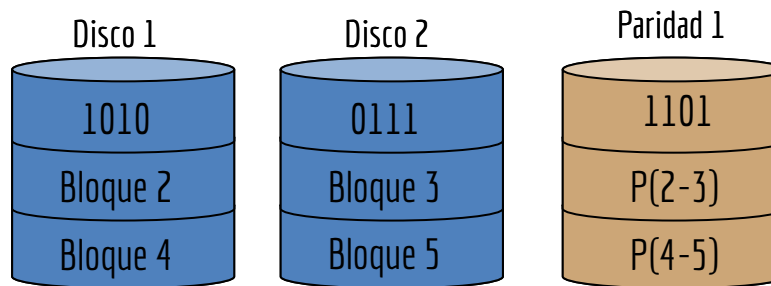
- Bandas de bits con código de corrección de errores (ECC) de Hamming
- Guardo el ECC en discos dedicados,
- Al escribir y al **leer** escribo el ECC
- Todos los discos giran sincronizados
- No permite requests paralelas a los discos ya que solo leo de a un bit
 - Hoy en día con sectores de 512 bytes en discos modernos no tiene sentido este nivel de RAID
 - Salvo usos específicos un request debería tener el tamaño de un sector o más

Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
Lento/Lento	-40%	N/2 Discos

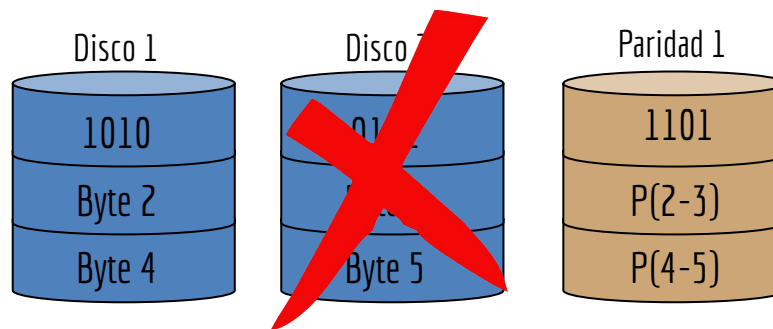


Paridad

- La calculo utilizando la operación XOR:

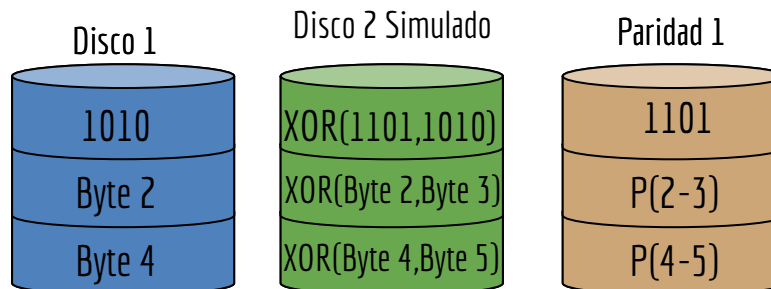


- En caso que **1** disco falle:



Input		Output
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

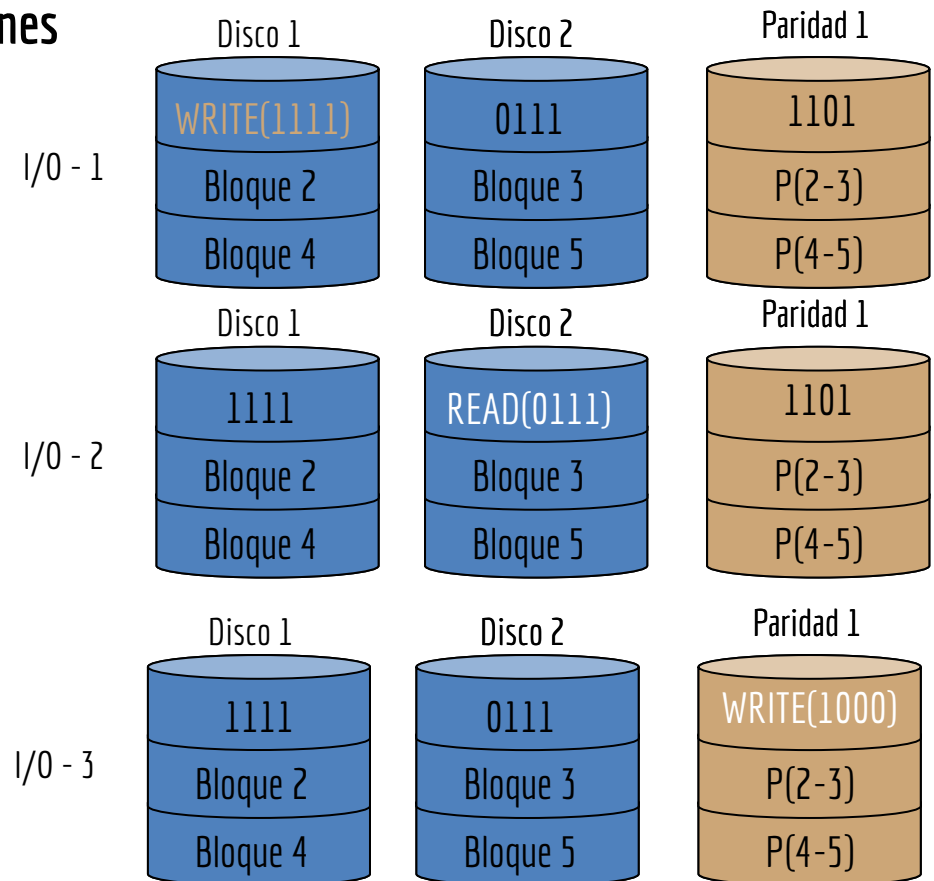
- Puedo seguir operando (pierdo performance hasta no reemplazarlo)



Escritura de Paridad (versión simple)

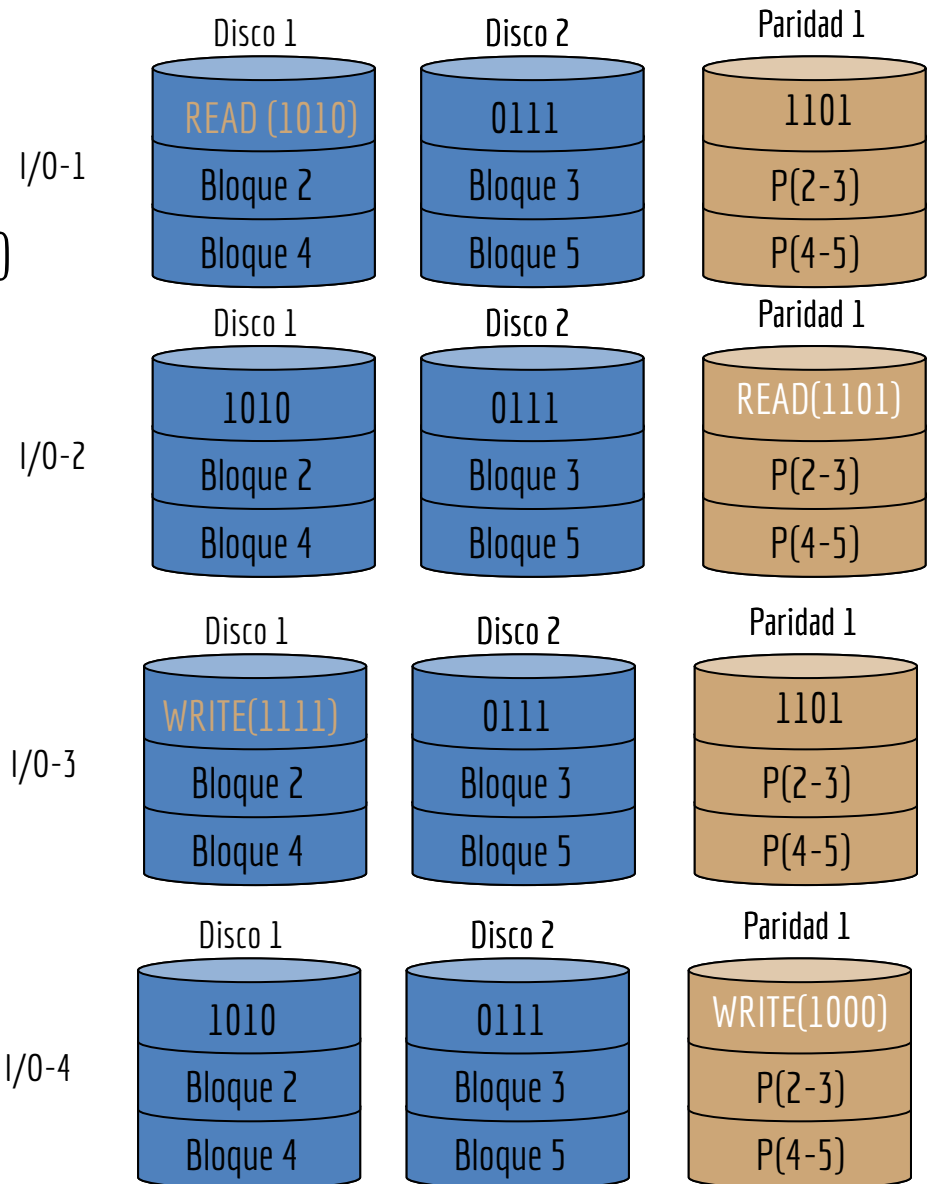
1. Quiero escribir **1111** en el bloque 0 (disco 1)
2. Escribo el nuevo dato
3. Leo de cada uno de los otros discos
 - a. Si tuviera n discos serian n operaciones más

4. Calculo $\text{delta} = \text{XOR}(\text{d_nuevos}, \text{Bloque 1})$ I/O - 1
5. Escribo p_nueva



Escritura de Paridad (Versión eficiente 4 I/O'S)

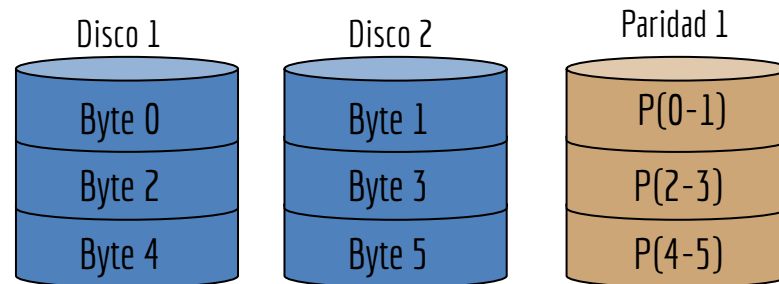
1. Quiero escribir **1111** en el bloque 0 (disco 1)
2. Leo los datos viejos **d_viejos = 1010**
3. Calculo **delta = XOR (d_viejos, d_nuevos)**
 - a. **delta = 0101**
4. Leo la paridad vieja **p_vieja = 1101**
5. Escribo el nuevo dato
6. Calculo **p_nueva = XOR (p_vieja, delta)**
 - a. **p_nueva = 1000**
7. Escribo **p_nueva**



RAID 3 (Striping de Bytes con paridad)

- Bandas de bytes con disco dedicado de paridad
- Al escribir escribo la paridad
- Todos los discos giran sincronizados
- No permite requests paralelas a los discos ya que solo leo de a un byte
 - Hoy en día con sectores de 512 bytes en discos modernos no tiene sentido este nivel de RAID
 - Salvo usos específicos un request debería tener el tamaño de un sector o más

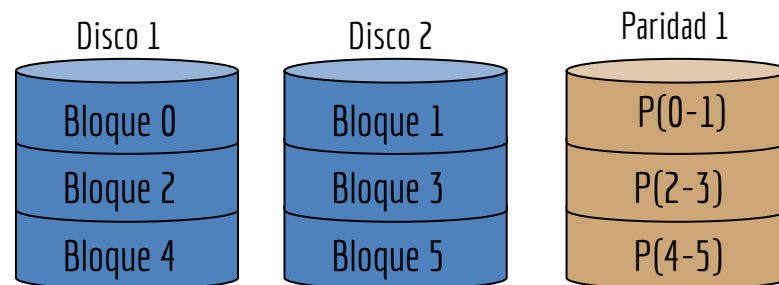
Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
Lento/Lento	N-1 Discos	1 Disco



RAID 4 (Bandas de Bloques con paridad)

- Al escribir escribo la paridad
- Velocidad Lectura de **n-1** discos
- Este esquema tiene más sentido que R2 y R3
- Teniendo **n** discos tengo **n-1** de capacidad
- Puedo perder 1 solo disco
- Existe un cuello de botella en que la paridad esté en un solo disco
- Requests de escrituras en paralelo sufren mucho en performance,
- En teoría la Vel. de E, parece ser buena, en la práctica la velocidad de escritura del siguiente nivel de Raid (R5) es mejor porque permite paralelismo

Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
$(N-1)X/NX/4$	N-1 Discos	1 Disco



Unidad 5 (Sistema Entrada/Salida)

* N es la cantidad de discos en RAID

* X es la velocidad de un solo disco

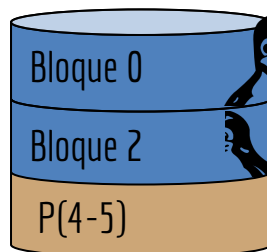
RAID 5

- Peor escritura que R10
- Velocidad Lectura de $n-1$ discos
- Teniendo n discos tengo $n-1$ de capacidad
- Puedo perder 1 solo disco
- **Distribuye la paridad para no bloquear los discos**
- Ideal datos no tan críticos donde se busca buena performance
- Overhead de calcular la paridad (4 operaciones $\Rightarrow NX/4$) [link](#)

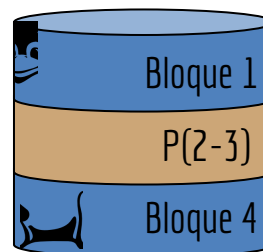


Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
$(N-1)X \mid NX/4$	- 1 Disco	1 Disco

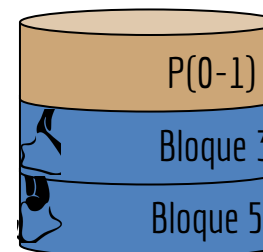
Disco 1



Disco 2



Disco 3

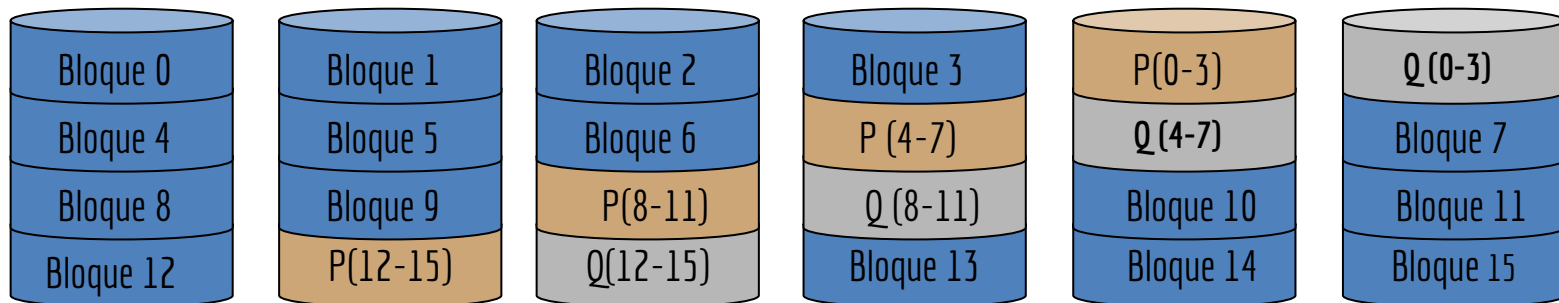


RAID 6

- Peor escritura que R5
- Teniendo **n** discos tengo **n-2** de capacidad
- Puedo perder 2 discos
- Velocidad Lectura de **n-2** discos
- Necesito placas controladoras especiales para RAID, no se puede por software
- Distribuir la paridad para no bloquear los discos
- Overhead de calcular la paridad (6 operaciones $\Rightarrow NX/6$) [link](#)



Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)
$(N-2)X \mid NX/6$	- 2 Discos	2 Discos



* N es la cantidad de discos en RAID

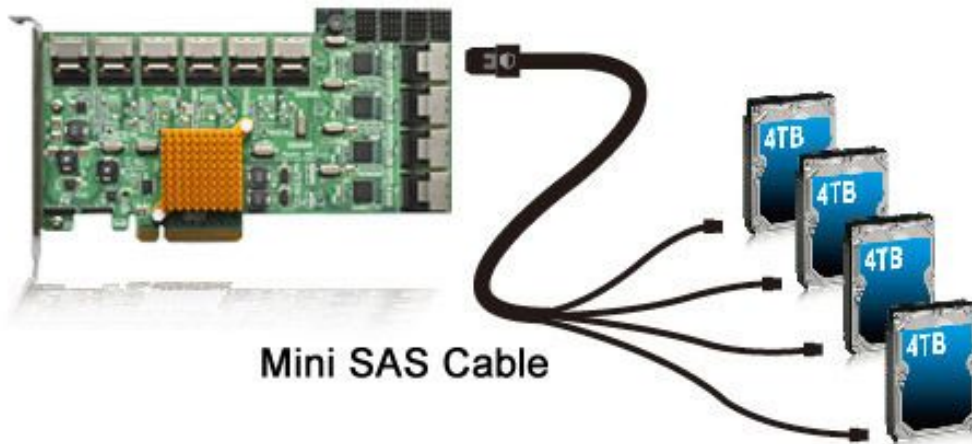
* X es la velocidad de un solo disco

NIVELES DE RAID

LVL	DESCRIPCIÓN	Veloci. L/E	-% Pérdida de Capacidad	Tolerancia a fallas (peor caso)	Aplicaciones Típicas
0	Bandas, alterno los bloques entre cada uno de los discos	$NX \mid NX$	0%	0 Discos	• SO's (velocidad) • Data logging • Rendering o caché
1	Espejo de un disco repetido n veces. La capacidad de solo uno	$NX \mid X$	n-1 Discos	n-1 Discos	• SO's (para redundancia) • DB's tradicionales • Backups domésticos
1+0	Bandas espejadas o espejos de bandas.	$NX \mid NX/2$	-50%	1 Disco	• Server de archivos • BD's veloces • Server de aplicaciones
2	Intercalación de Bits con redundancia por código Hamming	Mala	-40%	n/2 Discos	• Sin uso comercial
3	Paridad por intercalación de Bits	Mala	1 Disco	1 Disco	• Sin uso comercial
4	Paridad con intercalación de bloques	$(N-1)X \mid NX/4$	- 1 Disco	1 Disco	• Sin uso comercial • Aunque existen variantes similares
5	Paridad con intercalación distribuida de bloques	$(N-1)X \mid NX/4$	- 1 Disco	1 Disco	• Data warehouses • Web servers
6	Paridad con intercalación doblemente distribuida de bloques	$(N-2)X \mid NX/6$	- 2 Discos	2 Discos	• P/ soluciones de alta disponibilidad y capacidad • Backups comerciales • Archivamiento de datos

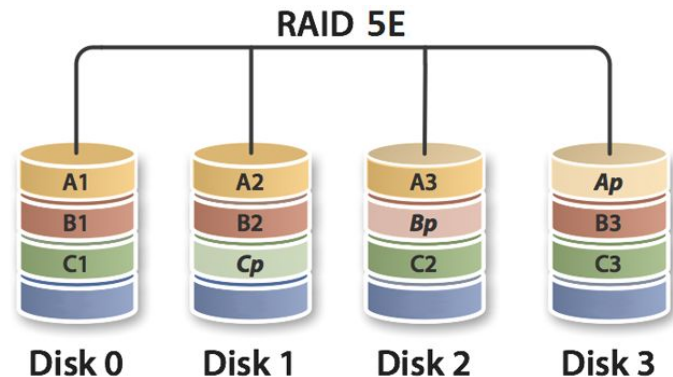
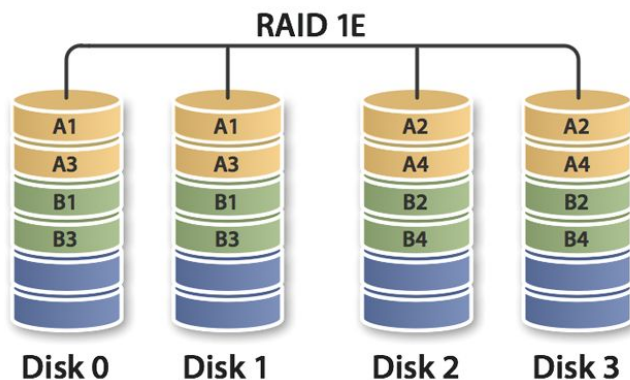
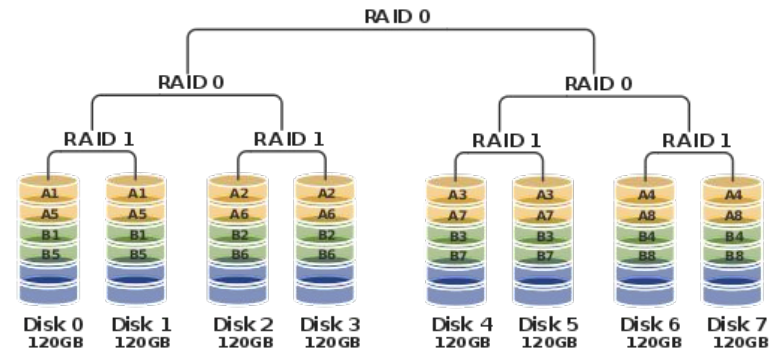
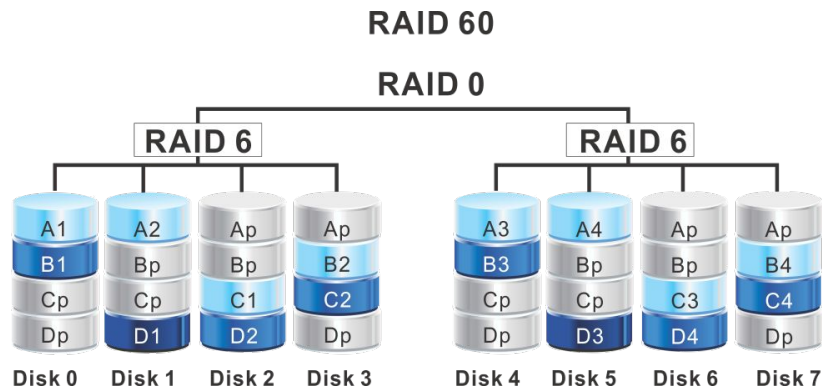
Tarjetas Controladoras

- Para utilizar cualquier nivel avanzado de RAID es recomendable o necesario usar hardware dedicado
- Estas tarjetas le sacan trabajo al CPU y se encargan de hacer calculo de paridad y reconstruir el array en caso de falla (**Hot spares**)
- SAS a x4 SATA3 6gbps



Más RAID?

- Existen combinaciones ([Nested RAID](#)) de las anteriores
- RAID 60, 50, 100, 1E, 5E ... y más



NON-RAID? La guerra de los NAS DIY

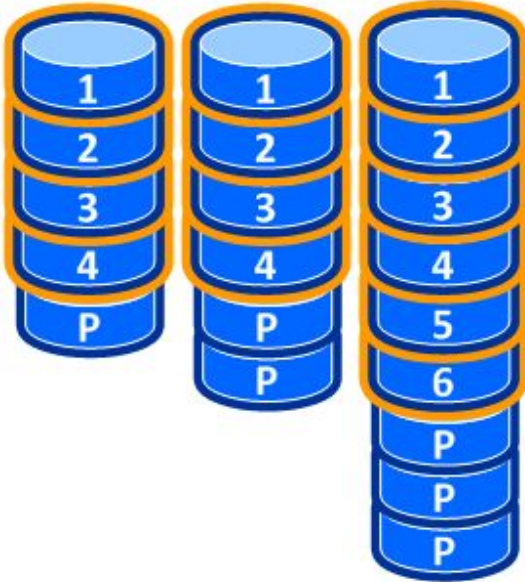


- Existen alternativas a RAID tradicional
- **FreeNAS** es una distro libre basada en FreeBSD que permite armar un NAS usando **OpenZFS**
 - **RAIDZ** divide el array en **ZPools** que utilizan el filesystem **ZFS** (Sun Microsystems, Oracle)
 - Cada **Zpool** puede tener su propio nivel de **RAID**, mientras que **ZFS** se encarga de que todo esté bajo el mismo FileSystem
 - Además **ZFS** usa la RAM como Caché*
 - **Hot spares**
 - Alta performance, enterprise
- **UNRAID** es una distro paga, también basada en FreeBSD, mas sencillo, usa **XFS**
 - **Permite** usar cualquier tipo de disco con cualquier hardware
 - Para principiantes, aficionados y pymes



* Requiere CPUs que soporten Ram **ECC** (Línea Xeon de Intel o Ryzen/TR/EPIC de AMD)

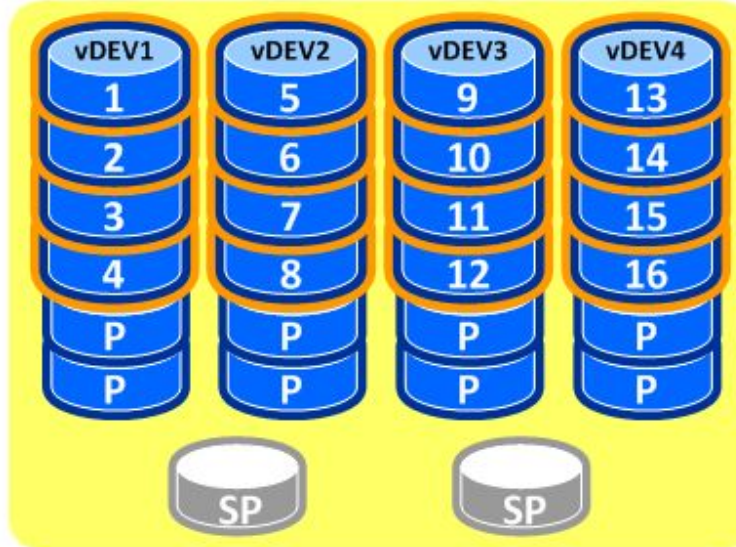
RAIDZ 5 disks RAIDZ-2 6 disks RAIDZ-3 9 disks



- RAIDZ1 => RAID 5
- RAIDZ2 => RAID 6
- RAIDZ3 => RAID 6 + 1 Paridad

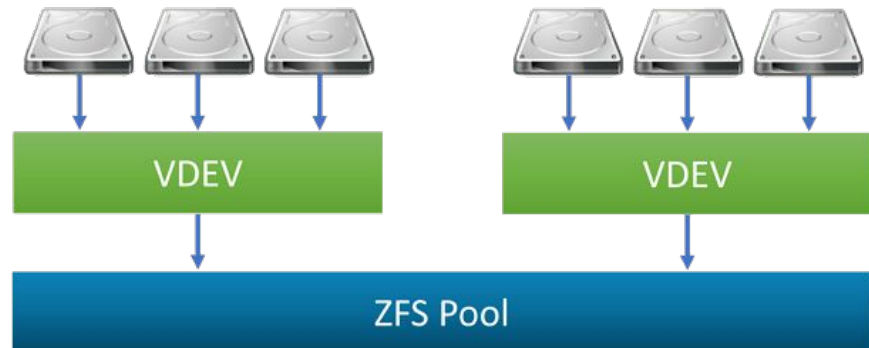
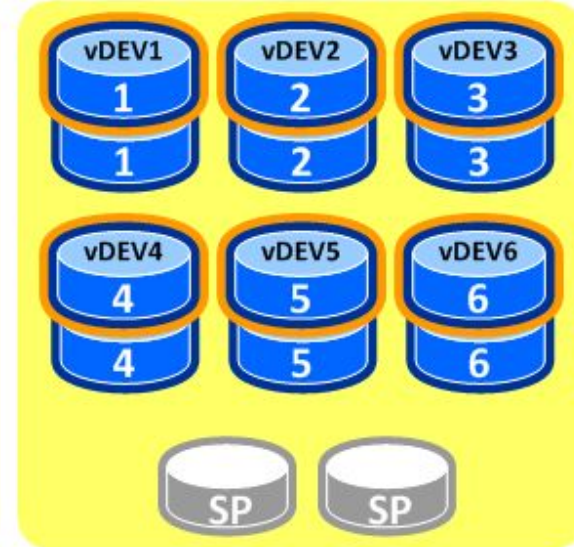
zPOOL => RAID 60

4vDEVs (RAIDZ-2 6 disks) wth 2 hotspares



zPOOL => RAID 10

8vDEVs mirrored disks with 2 hotspares





Hard Drive



Can be lost

Virtual Device (vdev)
I.E., a logical hard drive

If a single vdev loses more than the number of
allowed drives, the entire pools' data is lost.

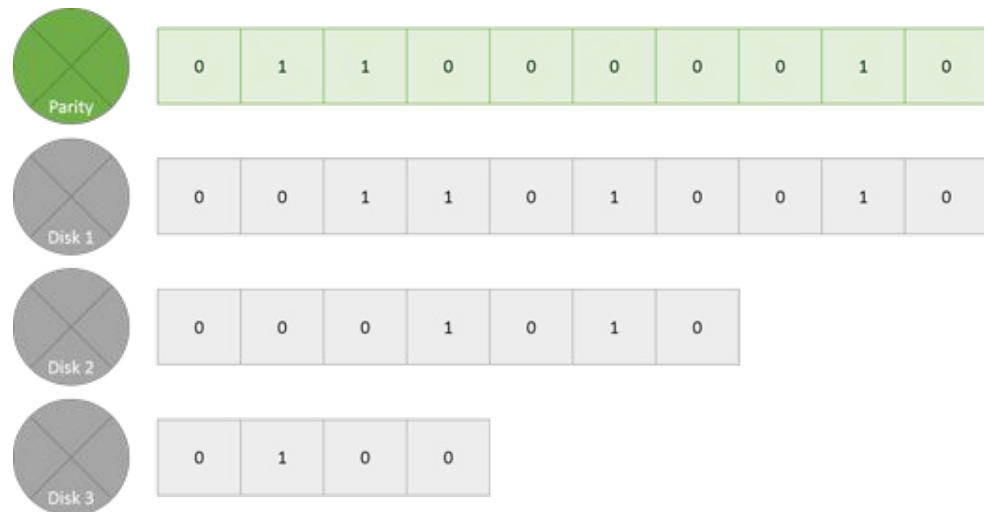
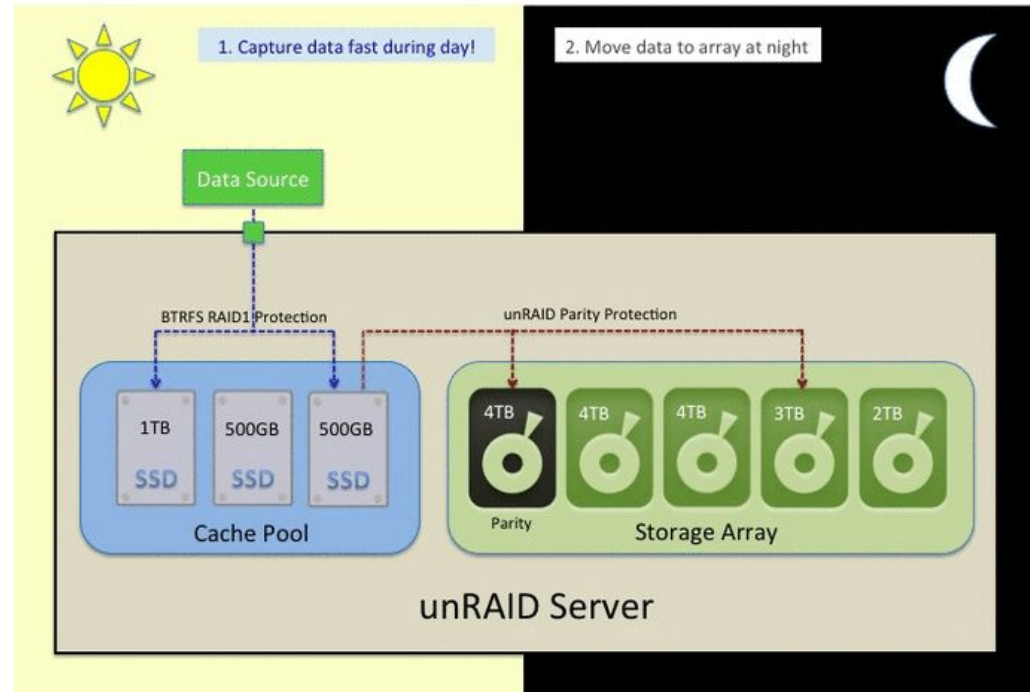




UNRAID

- **UNRAID (Pago)** en cambio usa otro esquema
- Expande la idea de [JBOD](#)
- Basado en **FreeBSD**
- Orientado a Archiving y facilidad de uso
- Permite mezclar discos de tamaños diferentes
- Usa 1 o 2 discos de paridad (símil RAID 4)
- Usa además 1 o 2 discos de Caché
- Combina al igual que ZFS un FileSystem (XFS)
- La unidad mínima de almacenamiento es 1 archivo, no hay striping
- Si se pierde información solo se pierde lo que estaba en ese disco y no en el resto

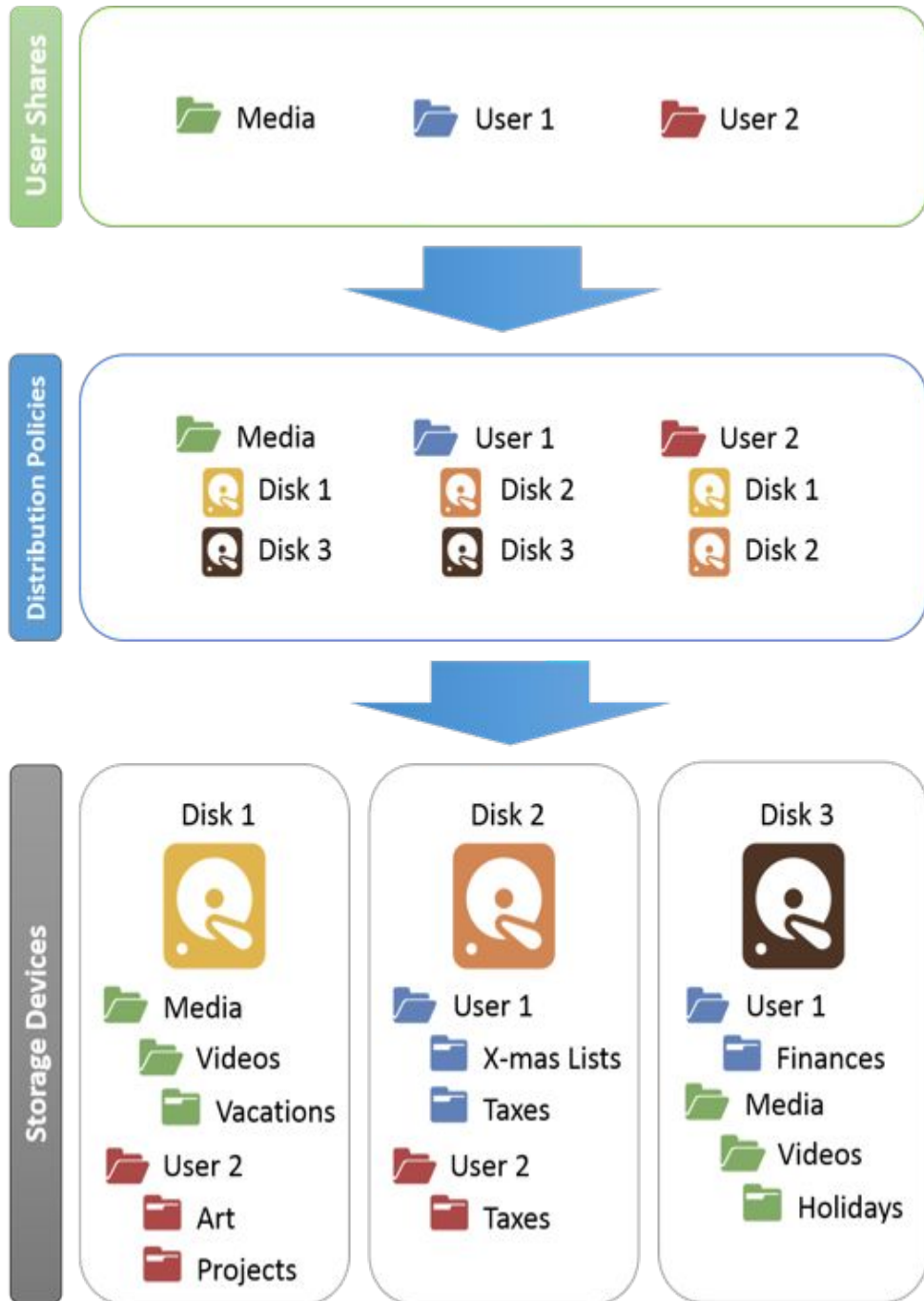
How Data Capture Works with Cache Pool



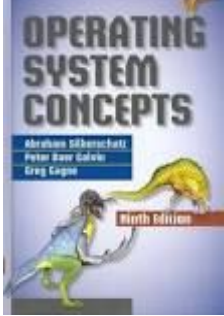


UNRAID

- **UNRAID (Pago)** en cambio usa otro esquema
- Expande la idea de [JBOD](#)
- Orientado a Archiving y facilidad de uso
- Permite mezclar discos de tamaños diferentes
- Usa 1 o 2 discos de paridad (símil RAID 4)
- Usa además 1 o 2 discos de Caché
- Combina al igual que ZFS un FileSystem (**XFS**)
- La unidad mínima de almacenamiento es 1 archivo, no hay striping
- Si se pierde información solo se pierde lo que estaba en ese disco y no en el resto



Referencias



- Cap. 10 - Operating System Concepts - Abraham Silberschatz
- Disk Scheduling Algorithms - <https://www.geeksforgeeks.org/disk-scheduling-algorithms/>
- RAID Types - <http://www.raid-calculator.com/raid-types-reference.aspx>
- RAID Performance - <https://blog.storagecraft.com/raid-performance/>